

Der Bentley-Ottmann Algorithmus

Ross Alexander Khrisna

Universität Kassel, Fachbereich 16 – Elektrotechnik/Informatik

1 Einleitung und Motivation

Geometrische Probleme spielen in der Informatik eine zentrale Rolle. Angefangen bei geografischen Informationssystemen, hin zu der Entwicklung von Microchips, bis zu CAD Anwendungen und der Analyse von Verkehrsnetzen. Grundlegend ist dabei die Fragestellung, ob und an welchen Stellen sich zwei oder mehrere Liniensegmente schneiden.

Durch die Kombination eines strukturierten Ordnungsbegriffs mit dynamischen Datenstrukturen ermöglichen sogenannte Sweep-Line-Algorithmen die Verarbeitung komplexer geometrischer Schnittprobleme in subquadratischer Laufzeit.

So ist der im Zentrum dieser Arbeit stehende Bentley-Ottmann-Algorithmus in der Lage alle Schnittpunkte zwischen n Segmenten mit einer Laufzeit von $\mathcal{O}((n + k) \log n)$, wobei k die Anzahl der tatsächlich auftretenden Schnittpunkte ist, zu bestimmen [2].

1.1 Ziel der Arbeit und Fragestellung

Das Ziel dieser Seminararbeit ist es die wichtigsten Konzepte, Methoden und Erkenntnisse des Bentley-Ottmann-Algorithmus verständlich und strukturiert aufzuarbeiten. Um dies zu erreichen, werden folgende Fragen beantwortet:

- Wie lassen sich geometrische Schnittprobleme formal beschreiben?
- Welche Grenzen haben naive Verfahren und warum reichen sie in vielen Anwendungen nicht aus?
- Wie funktioniert der Shamos-Hoey-Algorithmus als Vorläufer moderner Sweep-Line-Methoden?
- In welcher Weise erweitert und verbessert der Bentley-Ottmann-Algorithmus diesen Ansatz?
- Warum erreicht der Algorithmus die Laufzeit $\mathcal{O}((n + k) \log n)$?
- Welche Spezialisierungen und Verbesserungen existieren, und wie beeinflussen sie die praktische Anwendbarkeit?

1.2 Aufbau der Arbeit

Nach der Erarbeitung theoretischer Grundlagen zu Ordnungsbegriffen und der Laufzeitanalyse in Kapitel 2 führt diese Arbeit in Kapitel 3 in das Segment-Schnitt-Problem sowie dessen Anwendungskontexte ein. Den methodischen Kern bildet die Sweep-Line-Technik: Ausgehend vom Shamos-Hoey-Algorithmus in Kapitel 4 erfolgt in Kapitel 5 eine detaillierte Analyse des Bentley-Ottmann-Verfahrens hinsichtlich dessen Datenstrukturen und Komplexität. Darauf aufbauend werden in Kapitel 6 spezifische Optimierungen durch strukturelle Einschränkungen untersucht, bevor Kapitel 7 die Arbeit mit einer Reflexion der Ergebnisse sowie einem Ausblick auf weiterführende Fragestellungen abschließt.

2 Mathematische & theoretische Grundlagen

Sweep-Line-Algorithmen wie der Shamos-Hoey- oder der Bentley-Ottmann-Algorithmus basieren auf strukturierten Ordnungsbegriffen sowie dynamischen Datenstrukturen.

Diese werden während der Laufzeit des Algorithmus ständig aktualisiert, wodurch ein grundlegendes Verständnis von partiellen und totalen Ordnungen, Verbänden und Ordnungsdigrammen notwendig ist.

Dabei bildet die asymptotische Laufzeitanalyse das Fundament für die Bewertung der Effizienz der untersuchten Algorithmen.

2.1 Partielle und totale Ordnung

Definition 1 (Partiell geordnete Menge). *Ein Poset $(P, \leq)$ besteht aus einer Menge P und einer binären Relation $\leq \subseteq P \times P$, welche für alle $x, y, z \in P$ reflexiv ($x \leq x$), antisymmetrisch ($(x \leq y \wedge y \leq x) \implies x = y$) und transitiv ($(x \leq y \wedge y \leq z) \implies x \leq z$) ist [3].*

Elemente ohne Relation ($x \not\leq y \wedge y \not\leq x$) heißen *unvergleichbar* ($x \parallel y$). Gilt hingegen für alle Paare Vergleichbarkeit, liegt eine *totale Ordnung* vor.

Definition 2 (Totale Ordnung). *Ein Poset $(P, \leq)$ ist total geordnet, falls das Axiom der Totalität erfüllt ist [3]:*

$$\forall x, y \in P : (x \leq y) \vee (y \leq x) \quad (2.1)$$

2.2 Asymptotische Laufzeitanalyse

Die asymptotische Komplexität eines Algorithmus wird über die Landau-Symbole als Mengen von Funktionen klassifiziert [4].

Definition 3 (Asymptotische Schranken). *Für eine Funktion $g : \mathbb{N} \rightarrow \mathbb{R}^+$ sind die Schranken wie folgt definiert:*

- i) $\mathcal{O}(g(n)) = \{f(n) \mid \exists c, n_0 > 0 : \forall n \geq n_0 : 0 \leq f(n) \leq c \cdot g(n)\}$
- ii) $\Omega(g(n)) = \{f(n) \mid \exists c, n_0 > 0 : \forall n \geq n_0 : 0 \leq c \cdot g(n) \leq f(n)\}$
- iii) $\Theta(g(n)) = \mathcal{O}(g(n)) \cap \Omega(g(n))$

Diese Notationen erlauben die Worst-Case-Abgrenzung des Bentley-Ottmann-Algorithmus in Abhängigkeit der Segmentanzahl n und der Schnittpunktzahl k .

2.3 Binärer Suchbaum

Definition 4 (Balancierter binärer Suchbaum).

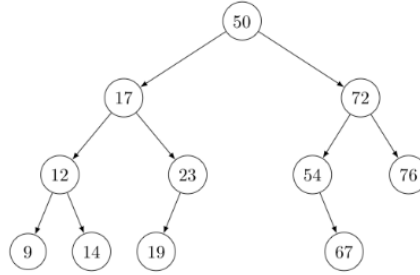


Abb. 2.1: Struktur eines balancierten binären Suchbaums (Quelle: [13]).

Ein balancierter binärer Suchbaum ist ein binärer Suchbaum mit n Knoten und Höhe h , für den gilt

$$h = \mathcal{O}(\log n).$$

Damit liegen die elementaren Operationen Suchen, Einfügen und Löschen im Worst Case in

$$\Theta(\log n).$$

2.4 Sortierung im Suchbaum

Die Statusstruktur des Sweep-Line-Verfahrens verwaltet eine totale Ordnung der aktiven Segmente basierend auf deren y -Koordinaten an der aktuellen Position x_{sw} .

Definition 5 (Segment-Funktion). Ein nicht-vertikales Segment s wird als lineare Funktion $y_s : [x_{s1}, x_{s2}] \rightarrow \mathbb{R}$ aufgefasst, wobei die Ordnung $\leq_{sw}$ zweier Segmente s, t definiert ist durch [5]:

$$s \leq_{sw} t \iff y_{s1} + (x_{sw} - x_{s1}) \cdot \frac{y_{s2} - y_{s1}}{x_{s2} - x_{s1}} \leq y_t(x_{sw}) \quad (2.2)$$

2.5 Orientierungstest

Der Orientierungstest (CCW-Test) bestimmt die relative Lage eines Punktes P_3 zu einer gerichteten Geraden durch P_1 und P_2 mittels Vorzeichenanalyse einer Determinante.

Definition 6 (Prädikat ccw). Für $P_1, P_2, P_3 \in \mathbb{R}^2$ ist die Orientierung über die Determinante der Matrix M definiert:

$$\text{ccw}(P_1, P_2, P_3) = \text{sgn} \left(\det \begin{pmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ 1 & x_3 & y_3 \end{pmatrix} \right) \in \{+1, -1, 0\} \quad (2.3)$$

Die Werte $+1, -1, 0$ korrespondieren zu einer Links- bzw. Rechtskurve oder Kollinearität.

Zwei Segmente $s = \overline{AB}$ und $t = \overline{CD}$ schneiden sich im Inneren, wenn ihre Endpunkte jeweils auf gegenüberliegenden Seiten der komplementären Trägergeraden liegen:

$$\text{ccw}(A, B, C) \cdot \text{ccw}(A, B, D) < 0 \wedge \text{ccw}(C, D, A) \cdot \text{ccw}(C, D, B) < 0 \quad (2.4)$$

Bei Kollinearität (alle $\text{ccw} = 0$) wird eine Überlappung über den Bounding-Box-Test der Koordinatenintervalle geprüft:

$$[\min(x_A, x_B), \max(x_A, x_B)] \cap [\min(x_C, x_D), \max(x_C, x_D)] \neq \emptyset \quad (2.5)$$

Dies stellt sicher, dass sich kollineare Segmente tatsächlich berühren oder überlappen.

2.6 Schnittpunktberechnung

Die Koordinaten des Schnittpunkts $P = (x, y)$ zweier sich schneidender Segmente lassen sich mittels der Cramer'schen Regel effizient bestimmen.

Definition 7 (Schnittpunktformel). Für nicht-parallele Segmente ($D \neq 0$) berechnen sich die Koordinaten über die Determinante D und die Zählermatrizen wie folgt:

$$D = (x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4) \quad (2.6)$$

$$x = \frac{(x_1 y_2 - y_1 x_2)(x_3 - x_4) - (x_1 - x_2)(x_3 y_4 - y_3 x_4)}{D} \quad (2.7)$$

$$y = \frac{(x_1 y_2 - y_1 x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 y_4 - y_3 x_4)}{D} \quad (2.8)$$

Dieser Ansatz vermeidet die explizite Berechnung von Geradensteigungen und erhöht somit die numerische Stabilität bei fast vertikalen Segmenten.

3 Das Segment-Schnittproblem

Gegeben ist eine Menge $S = \{s_1, \dots, s_n\}$ von n Liniensegmenten im $\mathbb{R}^2$. Ein Segment s_i ist definiert durch die Endpunkte $P_i, Q_i \in \mathbb{R}^2$ als die Menge aller Punkte:

$$s_i = \{P_i + \lambda(Q_i - P_i) \mid 0 \leq \lambda \leq 1\} \quad (3.1)$$

Das Segment-Schnitt-Problem wird je nach Anforderung formal wie folgt klassifiziert [1]:

- **Entscheidungsproblem:** Prüfe, ob $\exists s_i, s_j \in S, i \neq j : s_i \cap s_j \neq \emptyset$.
- **Reporting-Problem:** Bestimme die Menge aller Schnittpunkte $\mathcal{I} = \{p \in \mathbb{R}^2 \mid \exists s_i, s_j \in S, i \neq j : p \in s_i \cap s_j\}$.
- **Counting-Problem:** Bestimme die Kardinalität $|\mathcal{I}|$ der Menge der Schnittpunkte.

3.1 Naive Algorithmen und ihre Grenzen

Der naive Ansatz prüft jedes ungeordnete Paar $\{s_i, s_j\} \subseteq S$ auf Schnittpunkte. Bei $\binom{n}{2}$ Paaren ergibt sich eine Komplexität von $\mathcal{O}(n^2)$ [1].

Algorithm 1 Naiver Segment-Schnitt-Algorithmus

Eingabe: Menge $S = \{s_1, \dots, s_n\}$
Ausgabe: Menge der Schnittpunkte $\mathcal{I}$

```

1:  $\mathcal{I} \leftarrow \emptyset$ 
2: for jedes  $\{s_i, s_j\} \subseteq S$  mit  $i \neq j$  do
3:   if  $s_i \cap s_j \neq \emptyset$  then
4:      $P \leftarrow \text{berechneSchnittpunkt}(s_i, s_j)$ 
5:      $\mathcal{I} \leftarrow \mathcal{I} \cup \{P\}$ 
6:   end if
7: end for
8: return  $\mathcal{I}$ 
```

4 Shamos-Hoey-Algorithmus

Einer der ersten effizienten Algorithmen zur Erkennung von Segment-Schnittpunkten ist der Shamos-Hoey-Algorithmus [2].

Er basiert auf der Sweep-Line-Technik und erreicht eine Laufzeit von $\mathcal{O}(n \log n)$ [1].

Allerdings beschränkt sich das Ergebnis auf die Beantwortung des zuvor genannten Entscheidungsproblems.

4.1 Idee der Sweep-Line-Technik

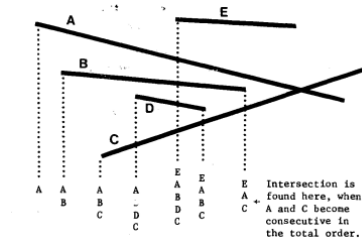


Abb. 4.1: Ermittlung von Schnittpunkten durch die Sweep-Line (Quelle: [1]).

Das Grundprinzip der Sweep-Line-Technik besteht darin, eine vertikale Linie von links nach rechts über die gesamte Ebene entlang der x -Achse zu bewegen. Dabei werden Segmente verwaltet und jeweils nur diejenigen Paare überprüft, die auf der Sweep-Line unmittelbar benachbart sind.

Während der Bewegung werden folgende Ereignisse vom Algorithmus verarbeitet:

1. **Linker Endpunkt eines Segments:** Hat die Sweep-Line den linken Endpunkt eines Segments erreicht, wird das Segment in den Sweep-Line-Status eingefügt.
2. **Rechter Endpunkt eines Segments:** Das Segment wird aus dem Sweep-Line-Status entfernt.

Zu jedem Zeitpunkt enthält der Sweep-Line-Status genau die Segmente, die die Sweep-Line schneidet. Diese Segmente werden nach ihrer y -Position auf der Sweep-Line geordnet.

4.2 Datenstrukturen und Ordnung der Segmente

Der Shamos-Hoey-Algorithmus benötigt zwei zentrale Datenstrukturen:

1. Event Queue

Eine Prioritätswarteschlange, die alle Segmentpunkte nach ihrer x -Koordinate sortiert beinhaltet. Die Eventqueue liefert die Ereignisse in der Reihenfolge, in der die Sweep-Line sie „sehen“ würde [1]. Wichtig sind dabei folgende Randfälle:

- (a) Bei gleichen x -Werten, wird der linke Endpunkt zuerst einsortiert.
- (b) Bei gleichen x -Werten vom selben Endpunkttyp, wird aufsteigend nach y sortiert.

2. Sweep-Line-Status

Ein binärer Suchbaum, der die aktuell aktiven Segmente speichert, geordnet nach ihrer y -Position auf der Sweep-Line [1]. Die Sortierung erfolgt gemäß Definition 5 und induziert zu jedem Zeitpunkt eine totale Ordnung im Sinne von Definition 2.

4.3 Laufzeitanalyse

Das Sortieren der Ereignisse bei $2n$ Segmentpunkten kostet $\mathcal{O}(n \log n)$.

Jedes Ereignis verursacht beim Einfügen oder Entfernen eines Segments in den Sweep-Line-Status eine Laufzeit von $\mathcal{O}(\log n)$, da dieser als balancierter binärer Suchbaum implementiert ist. Die Überprüfung der höchstens zwei unmittelbar benachbarten Segmente erfolgt in konstanter Zeit $\mathcal{O}(1)$ (vgl. Definitionen 4 und 3).

Da insgesamt $2n$ Ereignisse verarbeitet werden, ergibt sich eine Gesamtlaufzeit von $\mathcal{O}(n \log n)$ [1].

5 Der Bentley-Ottmann-Algorithmus

Der Bentley-Ottmann-Algorithmus gilt als Weiterentwicklung der Sweep-Line-Idee von Shamos und Hoey [2].

Während der Shamos-Hoey-Algorithmus lediglich entscheidet, ob ein Schnittpunkt existiert, löst Bentley-Ottmann das umfassende Reporting- und Counting Problem [2].

5.1 Problem: Report aller Schnittpunkte

Das Problem ist anspruchsvoller als das reine Entscheidungsproblem, da bei n Segmenten die Anzahl der Schnittpunkte k quadratisch in n wachsen kann. Daraus resultiert eine Anzahl von $k = \mathcal{O}(n^2)$ Schnittpunkten. Zusätzlich muss beim überqueren eines Schnittpunktes die Sweep-Line-Reihenfolge angepasst werden, wodurch weitere potentielle Schnittpunkte auftreten können. Daher erweitert der Bentley-Ottmann-Algorithmus die Sweep-Line-Technik um ein weiteres Ereignis [2]: **Schnittpunkt zweier aktiver Segmente**. Der Algorithmus muss somit zusätzlich garantieren, dass jeder Schnittpunkt genau einmal entdeckt wird, die Sweep-Line-Ordnung durch eine Prioritätswarteschlange korrekt aktualisiert wird und keine unnötigen Vergleiche stattfinden.

5.2 Event-Menge

Wie bereits erwähnt erzeugt der Bentley-Ottmann-Algorithmus neue Events dynamisch. Wird ein neues Segment in den Sweep-Line-Status eingefügt, so wird zunächst überprüft, ob es sich mit seinen direkten Nachbarn schneidet. Die Entscheidung erfolgt nach Definition 6. Falls ja, wird deren Schnittpunkte berechnet (vgl. Definition 7) und ein neues Event erzeugt. Dieses neue Event wird der Prioritätswarteschlange hinzugefügt (falls nicht schon vorhanden).

5.3 Sweep-Line-Status als totale Ordnung

Der Sweep-Line-Status unterstützt die folgenden Operationen jeweils in Worst-Case-Zeit $\mathcal{O}(\log n)$ (vgl. Definition 4):

- $\text{INSERT}(s)$ – Segment tritt in die Sweep-Line ein
- $\text{DELETE}(s)$ – Segment verlässt die Sweep-Line
- $\text{PRED}(s)$ – oberer Nachbar von s
- $\text{SUCC}(s)$ – unterer Nachbar von s
- $\text{SWAP}(s, t)$ – Austausch zweier Segmente an einem Schnittpunkt

5.4 Pseudocode

Der Bentley-Ottmann-Algorithmus nutzt eine Event-Queue Q und einen binären Suchbaum R , um Schnittpunkte effizient zu detektieren [2].

Algorithm 2 Bentley-Ottmann-Algorithmus

Require: Menge von Segmenten S

Ensure: Menge der Schnittpunkte $\mathcal{I}$

```

1:  $Q \leftarrow$  Prioritätswarteschlange aller Endpunkte von  $S$ 
2:  $R \leftarrow$  leerer Suchbaum (Sweep-Line-Status)
3:  $\mathcal{I} \leftarrow \emptyset$ 
4: while  $Q$  ist nicht leer do
5:    $p \leftarrow Q.\text{popMin}()$  ▷ Nächstes Ereignis nach  $x$ -Koordinate
6:   if  $p$  ist linker Endpunkt von  $s$  then
7:      $R.\text{insert}(s)$ 
8:      $\text{findeSchnitt}(s, R.\text{above}(s), Q)$ 
9:      $\text{findeSchnitt}(s, R.\text{below}(s), Q)$ 
10:  end if
11:  if  $p$  ist rechter Endpunkt von  $s$  then
12:     $s_{\text{up}} \leftarrow R.\text{above}(s)$ ;  $s_{\text{low}} \leftarrow R.\text{below}(s)$ 
13:     $R.\text{remove}(s)$ 
14:     $\text{findeSchnitt}(s_{\text{up}}, s_{\text{low}}, Q)$ 
15:  end if
16:  if  $p$  ist Schnittpunkt von  $s$  und  $t$  then
17:     $\mathcal{I} \leftarrow \mathcal{I} \cup \{p\}$ 
18:     $R.\text{swap}(s, t)$ 
19:     $\text{findeSchnitt}(s, R.\text{neighbor}(s), Q)$ 
20:     $\text{findeSchnitt}(t, R.\text{neighbor}(t), Q)$ 
21:  end if
22: end while
23: return  $\mathcal{I}$ 

```

Die Effizienz basiert auf der dynamischen Ereigniserzeugung, bei der neu erkannte Schnittpunkte sofort in Q fließen (Z. 12, 13, 17, 21, 22). Die lokale Prüfung beschränkt Vergleiche auf adjazente Nachbarn beim Einfügen (Z. 12–13), Entfernen (Z. 17) oder Tauschen (Z. 21–22). Dabei sichert der Tausch (Z. 20) die Ordnungskonsistenz des binären Suchbaums R für alle nachfolgenden Operationen.

5.5 Laufzeitanalyse

Der Bentley–Ottmann-Algorithmus besitzt eine Gesamtlaufzeit von $\mathcal{O}((n + k) \log n)$ [2], wobei k die Anzahl der tatsächlich auftretenden Schnittpunkte bezeichnet. Diese Komplexität lässt sich durch die Analyse der Elementaroperationen auf den Datenstrukturen Q (Event-Queue) und R (Suchbaum) beweisen.

A) Mächtigkeit der Ereignismenge Die Menge aller Ereignisse E ist die Vereinigung der Endpunkte P, Q und der Menge der Schnittpunkte $\mathcal{I}$. Für die Kardinalität $|E|$ der Prioritätswarteschlange gilt:

$$|E| = |P| + |Q| + |\mathcal{I}| = n + n + k = 2n + k = \mathcal{O}(n + k) \quad (5.1)$$

Da jedes Ereignis genau einmal aus Q extrahiert und verarbeitet wird, ist die Anzahl der Schleifendurchläufe durch $\mathcal{O}(n + k)$ beschränkt.

B) Aufwand pro Ereignis Jedes Ereignis $p \in E$ induziert eine konstante Anzahl an Operationen im Suchbaum R (Einfügen, Entfernen, Tauschen) sowie auf der Event-Queue Q (Einfügen neuer Schnittpunkte). Da $|R| \leq n$ und $|Q| = \mathcal{O}(n + k)$, beträgt der Zeitaufwand pro Ereignis:

$$T_{event} = \mathcal{O}(\log |R| + \log |Q|) = \mathcal{O}(\log n + \log(n + k)) = \mathcal{O}(\log n) \quad (5.2)$$

C) Beschränkung der lokalen Prüfungen Die Anzahl der geometrischen Schnittpunkte C ist linear zur Anzahl der Ereignisse, da pro Event-Typ nur eine konstante Anzahl an Nachbarschaften geprüft wird:

- **Einfügen (Linker Endpunkt):** Maximal 2 Prüfungen mit dem oberen und unteren Nachbarn.
- **Entfernen (Rechter Endpunkt):** Maximal 1 Prüfung zwischen den nun adjazenten Nachbarn.
- **Tausch (Schnittpunkt):** Maximal 2 Prüfungen mit den neuen Nachbarn der getauschten Segmente.

Daraus folgt $C \leq 2n + n + 2k = \mathcal{O}(n + k)$. Die Gesamtlaufzeit ergibt sich somit zu $\mathcal{O}((n + k) \log n)$.

Beweis der Optimalität Die Schranke $\Omega((n + k) \log n)$ ist für das Reporting-Problem optimal, da:

- i) das initiale Sortieren der Endpunkte bereits $\Omega(n \log n)$ erfordert,
- ii) die explizite Ausgabe von k Schnittpunkten eine Zeit von $\Omega(k)$ bedingt,
- iii) jede Lokalisierung im dynamischen Suchbaum R eine informationstheoretische Untergrenze von $\Omega(\log n)$ pro Operation besitzt [1].

Obwohl der Bentley-Ottmann-Algorithmus sehr effizient ist, existieren heute schnellere Methoden. So konnten Chazelle und Edelsbrunner (1992) [7] den logarithmischen Schritt ($O(\log n)$) pro Schnittpunkt auf ein Minimum reduzieren und eine Laufzeit von $O(n \log n + k)$ erreichen. Balaban (1995) [8] verbesserte dies noch weiter, indem er eine Lösung fand, die zusätzlich deutlich weniger Arbeitsspeicher benötigt.

6 Spezialisierungen und Verbesserungen

In vielen praktischen Anwendungen treten zusätzliche Strukturinformationen auf, die eine weitere Optimierung ermöglichen.

6.1 Horizontale und vertikale Segmente

Ein in der Praxis (z. B. CAD-Systeme, VLSI-Design) häufig auftretender Spezialfall ist die Menge der orthogonalen Segmente $S = H \cup V$, wobei H die Menge der horizontalen und V die Menge der vertikalen Segmente beschreibt [2]. Diese Mengen sind formal definiert als:



Abb. 6.1: Schnittpunkte in einer Menge orthogonaler Segmente (Quelle: [14]).

$$H = \{s_i \mid s_i = [(x_{i,1}, y_i), (x_{i,2}, y_i)], x_{i,1} < x_{i,2}\} \quad (6.1)$$

$$V = \{s_j \mid s_j = [(x_j, y_{j,1}), (x_j, y_{j,2})], y_{j,1} < y_{j,2}\} \quad (6.2)$$

Unter der Annahme der allgemeinen Lage (keine kollinearen Überlappungen) ergeben sich folgende geometrische Restriktionen:

- **Disjunktheit innerhalb der Klassen:** Für zwei beliebige Segmente s_a, s_b aus der gleichen Menge (H oder V) gilt $s_a \cap s_b = \emptyset$.
- **Intersektionsbedingung:** Ein Schnittpunkt $P = (x_P, y_P)$ existiert genau dann zwischen $s_h \in H$ und $s_v \in V$, wenn gilt:

$$x_{h,1} \leq x_v \leq x_{h,2} \quad \wedge \quad y_{v,1} \leq y_h \leq y_{v,2} \quad (6.3)$$

Durch diese strukturelle Einschränkung reduziert sich der Suchraum für potenzielle Schnittpunkte massiv. Während im allgemeinen Fall jedes Paar betrachtet werden müsste, beschränkt sich das Problem hier auf die Prüfung der Kreuzungspunkte zwischen den Mengen H und V . Die Anzahl der Vergleiche sinkt somit von $\mathcal{O}(n^2)$ auf $\mathcal{O}(|H| \cdot |V|)$. Mit spezialisierten Datenstrukturen wie Range-Trees oder optimierten Sweep-Line-Verfahren lässt sich dieses Problem sogar in $\mathcal{O}(n \log n + k)$ lösen.

6.2 Numerische Stabilität und praktische Umsetzung

In der theoretischen Betrachtung wird oft von exakten Koordinaten ausgegangen. In der Praxis führen jedoch Gleitkomma-Berechnungen bei fast parallelen Segmenten oder sehr nah beieinander liegenden Schnittpunkten zu Rundungsfehlern. Diese können die Ordnung im Sweep-Line-Status zerstören.

Moderne Standard-Bibliotheken wie CGAL (Computational Geometry Algorithms Library) [11] lösen dieses Problem, indem sie den Bentley-Ottmann-Ansatz mit exakter Arithmetik kombinieren. Dabei werden kritische Entscheidungen (wie der Orientierungstest in Definition 6) fehlerfrei berechnet. Ein bewährtes Verfahren zur Stabilisierung ist das sogenannte Snap Rounding [10]. Hierbei werden alle Schnittpunkte auf ein festes Koordinatengitter gezwungen. Dies stellt sicher, dass die Beziehungen der Segmente auch bei begrenzter Rechenpräzision erhalten bleiben und die Software (etwa die Implementierung in CGAL [12]) robust gegenüber extremen Spezialfällen wird.

7 Fazit und Ausblick

Der Shamos-Hoey-Algorithmus zeigte, dass das reine Entscheidungsproblem in $\mathcal{O}(n \log n)$ lösbar ist. Der Bentley-Ottmann-Algorithmus erweitert diese Idee zum vollständigen Reporting-Verfahren, das alle Schnittpunkte in einer Laufzeit von $\mathcal{O}((n+k) \log n)$ bestimmt und damit optimal arbeitet. Wesentliche Elemente dieser Effizienz sind die dynamische totale Ordnung der Segmente, die lokale Prüfung nur benachbarter Segmente sowie die Behandlung von End- und Schnittpunkten als Ereignisse.

7.1 Grenzen des Bentley-Ottmann-Algorithmus

Trotz der optimalen Laufzeit besitzt der Algorithmus einige Einschränkungen. Zum einen dominiert die Ausgabezeit den gesamten Ablauf, wenn die Anzahl der Schnittpunkte k quadratisch wächst. Zum anderen ist die Implementationskomplexität zu berücksichtigen, da der Algorithmus deutlich komplexer als naive oder lineare Sweep-Ansätze ist und ausgefeilte Datenstrukturen erfordert.

7.2 Mögliche weiterführende Themen

Im Rahmen zukünftiger Arbeiten bieten sich verschiedene Vertiefungen an, die über die hier behandelte deterministische Sweep-Line hinausgehen. Von besonderem Interesse sind randomisierte inkrementelle Konstruktionen, wie sie etwa von Mulmuley [9] eingeführt wurden. Diese verzichten auf die starre Ordnung einer Sweep-Line und bieten in der Praxis oft eine bessere Performance bei gleichzeitig einfacherer Implementierung. Darüber hinaus stellen die Parallelisierung von Sweep-Line-Methoden sowie die Erweiterung auf komplexe Geometrien wie Polygone, Kurven oder 3D-Modelle spannende Forschungsfelder dar. Diese

Entwicklungen unterstreichen, dass die effiziente Lösung geometrischer Schnittprobleme auch Jahrzehnte nach Bentley und Ottmann ein hochaktuelles Thema der Informatik bleibt.

Literatur

- [1] M. I. Shamos and D. J. Hoey, “Geometric intersection problems,” in Proc. 17th Annu. Conf. Foundations of Computer Science, pp. 208–215, Oct. 1976.
- [2] J. L. Bentley and T. A. Ottmann, “Algorithms for Reporting and Counting Geometric Intersections,” IEEE Transactions on Computers, Vol. C-28, No. 9, pp. 643–647, 1979.
- [3] L. Levine, Lecture Notes on Partially Ordered Sets, Algebraic Combinatorics, 2011.
- [4] D. E. Knuth, “Big Omicron and Big Omega and Big Theta,” SIGACT News, Vol. 8, No. 2, Apr.–June 1976, pp. 18–24.
- [5] Kathrin Langens, Interpolation von Daten, Ausarbeitung zum Hauptseminar “Mathematische Modellierung” (SoSe 2019), Johannes-Gutenberg-Universität Mainz, 2019,
https://www.numerik.mathematik.uni-mainz.de/files/2019/06/Langens_Kathrin_Ausarbeitung.pdf, Stand: 08.01.2026.
- [6] R. Sedgewick and K. Wayne, Algorithms, 4th Edition – Section 9.1: Geometric Primitives, Princeton University,
<https://algs4.cs.princeton.edu/91primitives/>, Stand: 08.01.2026.
- [7] B. Chazelle and H. Edelsbrunner, “An optimal algorithm for intersecting line segments in the plane,” Journal of the ACM (JACM), Vol. 39, No. 1, pp. 1–54, 1992.
- [8] I. J. Balaban, “An optimal algorithm for finding segments intersections,” in Proc. 11th Annu. Symposium on Computational Geometry (SoCG ’95), pp. 211–219, 1995.
- [9] K. Mulmuley, “A Fast Planar Partition Algorithm, I,” Journal of Symbolic Computation, Vol. 10, No. 3-4, pp. 253–280, 1990.
- [10] J. D. Hobby, “Practical segment intersection with finite precision output,” Computational Geometry, Vol. 13, No. 4, pp. 199–214, 1999.
- [11] The CGAL Project, CGAL User and Reference Manual, CGAL Editorial Board, 6.1.1 edition, 2026,
<https://doc.cgal.org/latest/Manual/packages.html>.
- [12] E. Packer, “2D Snap Rounding,” in CGAL User and Reference Manual, CGAL Editorial Board, 6.1.1 edition, 2026,
https://doc.cgal.org/latest/Snap_rounding_2/index.html.
- [13] Oxford College of Emory University, “Binary Search Trees,”
<https://mathcenter.oxford.emory.edu/site/cs171/binarySearchTrees/>,
 Stand: 04.02.2026.
- [14] J. Denning, “BST Geometry: Orthogonal Segment Intersection,” Taylor University,
https://cse.taylor.edu/~jdenning/classes/cos265/slides/10_BSTGeometry.html,
 Stand: 04.02.2026.